

Auto Sales Data Analysis

Project Report

Cover Page

Project Title: Exploratory and Analytical Data Analysis on Auto Sales Dataset

Submitted By: [Aditya Karmakar]

Course/Department: [B.Tech CSE]

Institution Name: [Lovely Professional University]

Date: [12-04-2025]

Declaration

I hereby declare that the project titled "*Exploratory and Analytical Data Analysis on Auto Sales Dataset*" is the result of my independent research and analysis conducted over the course of this academic term. I confirm that the dataset used for this project is available in the public domain and has not been altered beyond necessary preprocessing for analysis. Any references to existing work, theories, tools, or third-party libraries have been cited appropriately in the references section.

Signature:



Name: [Aditya Karmakar]

Certificate

This is to certify that the project entitled "*Exploratory and Analytical Data Analysis on Auto Sales Dataset*" submitted by **[Your Name]**, a student of **[Your Course/Institution]**, has been thoroughly reviewed and found to meet the academic requirements of the institution. The student has demonstrated strong analytical and visualization skills and has effectively communicated insights through this project.

Supervisor's Name: [Supervisor Name]

Designation: [Designation]

Date: [Date]

Acknowledgement

I am deeply grateful to my guide and mentor, **[Supervisor Name]**, for their unwavering support, encouragement, and expert guidance throughout the duration of this project. I extend my appreciation to my faculty, peers, and well-wishers who provided feedback and shared their

experiences with data analysis. Lastly, I would like to acknowledge the creators and maintainers of the dataset, which served as the foundation of this analytical exploration.

Table of Contents

- 1. Introduction
 - 2. Source of Dataset
 - 3. EDA Process
 - 4. Analysis on Dataset
 - Sales Trend Analysis
 - Product and Category Performance
 - Customer Segmentation and Behavior
 - Pricing Strategy and Discount Impact
 - Order Fulfillment and Status Impact
 - 5. Conclusion
 - 6. Future Scope
 - 7. References
-

1. Introduction

In today’s competitive global market, businesses in the automotive sector must constantly adapt to changes in customer preferences and economic conditions. Data analytics plays a pivotal role in enabling informed decision-making by uncovering hidden patterns, trends, and relationships within business data. The objective of this project is to utilize a structured exploratory and analytical approach to extract insights from an auto sales dataset. This report highlights the methodology followed, analytical techniques applied, and business interpretations derived from the data.

2. Source of Dataset

The dataset used for this project, titled *"Auto Sales data.csv"*, is publicly accessible and contains comprehensive transactional data, including customer orders, product information, sales values, geographical distribution, deal sizes, and order statuses. The dataset spans multiple years and encompasses both high-level and granular sales attributes, making it suitable for thorough exploratory data analysis and business intelligence reporting.

3. EDA Process

The Exploratory Data Analysis (EDA) was performed using Python and Jupyter Notebook as the development environment. The main steps in the EDA process were:

- Importing core libraries such as Pandas for data manipulation, NumPy for numerical operations, Matplotlib and Seaborn for visualization, and others like Squarify for advanced charting.
 - Reading and loading the dataset into a DataFrame using `pd.read_csv()` and performing structural inspection using `df.head()`, `df.info()`, and `df.describe()`.
 - Converting relevant date columns to datetime format and creating derived attributes like Year, Month, and Quarter.
 - Identifying and handling missing data, data type mismatches, and duplicate entries.
 - Performing grouping and aggregations to understand the dataset's shape and structure.
 - Creating visual representations of sales trends, category breakdowns, and customer behavior using bar plots, line plots, scatter plots, and heatmaps.
-

4. Analysis on Dataset

4.1 Sales Trend Analysis

i. Introduction: This section focuses on identifying patterns in total sales over time by examining trends across months and years.

ii. General Description: By extracting year and month components from order dates, the dataset is grouped and aggregated to calculate monthly and yearly sales totals. This allows the identification of high and low-performing time periods.

iii. Specific Requirements, Functions, and Formulas:

- Pandas functions such as `groupby`, `dt.year`, `dt.month`, and `sum()`
- Seaborn's `lineplot()` to visualize sales trends over time

iv. Analysis Results: The analysis revealed seasonal patterns in sales, with notable peaks during November and December, likely due to end-of-year promotions and buying behavior. Sales typically dip mid-year, providing opportunities for targeted marketing.

v. Visualization: Line plots comparing sales trends across different years, grouped by month.

4.2 Product and Category Performance

i. Introduction: This section investigates which product lines and individual products contribute most significantly to revenue generation.

ii. General Description: Aggregating sales by `PRODUCTLINE` and `PRODUCTCODE` identifies top-performing categories and items. Comparing average sales across these groups sheds light on the product mix's effectiveness.

iii. Specific Requirements, Functions, and Formulas:

- `groupby`, `sum()`, and `mean()` to compute total and average sales per product line
 - Visualization using `barplot()` and optional `treemap()` for hierarchical display
- iv. Analysis Results:** The results indicated that a few product lines, such as Classic Cars and

Vintage Cars, were responsible for the bulk of revenue. A small set of SKUs consistently generated the highest sales.

v. Visualization: Vertical bar charts of revenue by category and top products by total sales.

4.3 Customer Segmentation and Behavior

i. Introduction: To optimize marketing efforts and customer retention strategies, this section explores customer contributions by geography and order size.

ii. General Description: By segmenting the data based on COUNTRY, CUSTOMERNAME, and DEALSIZE, we identify high-value customers and regions. Frequency and volume metrics are also used to categorize customers.

iii. Specific Requirements, Functions, and Formulas:

- Pandas value_counts(), groupby, and agg()
- Seaborn's barplot() and countplot() for frequency distribution

iv. Analysis Results: The findings showed that a small group of customers contributed disproportionately to revenue, supporting the Pareto principle. Countries such as the USA and France led in total sales.

v. Visualization: Bar plots of customer revenue contribution and regional performance.

4.4 Pricing Strategy and Discount Impact

i. Introduction: This section evaluates how unit price and discounting affect order quantity and total revenue.

ii. General Description: Analyzing correlations between PRICEEACH, QUANTITYORDERED, and SALES provides insights into pricing elasticity and customer responsiveness.

iii. Specific Requirements, Functions, and Formulas:

- Use of corr() for correlation matrix
- Binning prices using cut() and plotting relationships with scatterplot() and heatmap()

iv. Analysis Results: The analysis demonstrated that while lower prices correlated with higher quantities ordered, total revenue was optimized by balancing pricing and volume.

v. Visualization: Correlation heatmaps, scatter plots, and binned price-revenue charts.

4.5 Order Fulfillment and Status Impact

i. Introduction: Evaluates the influence of order statuses on revenue and identifies bottlenecks in the fulfillment process.

ii. General Description: Categorizing data by STATUS allows understanding of the proportion of successful versus cancelled orders. It also helps assess logistics efficiency.

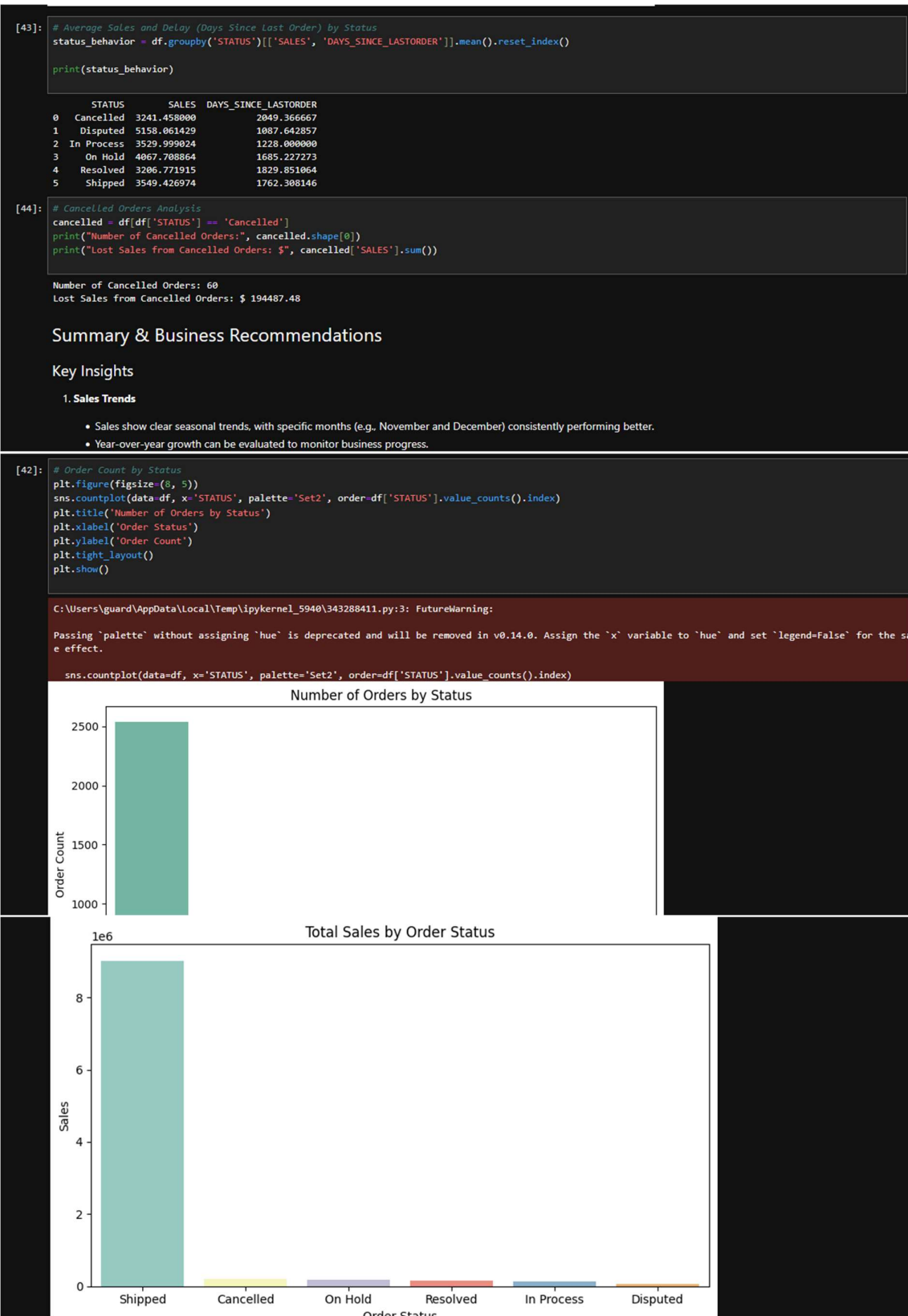
iii. Specific Requirements, Functions, and Formulas:

- Grouping using groupby('STATUS'), calculating count() and sum()
- Visualizing using Seaborn's barplot() and countplot()

iv. Analysis Results: Orders marked as "Shipped" contributed overwhelmingly to revenue, while "Cancelled" and "On Hold" orders represented opportunity costs.

v. Visualization: Bar charts displaying order status distribution and corresponding revenue impacts.

Images:



Objective 5: Order Fulfillment and Status Impact

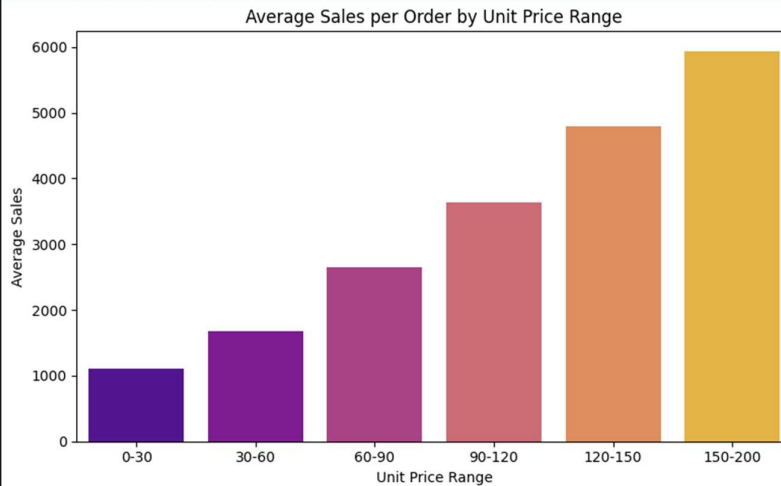
```
[41]: # Sales Breakdown by Order Status
status_sales = df.groupby('STATUS')['SALES'].sum().reset_index().sort_values(by='SALES', ascending=False)

plt.figure(figsize=(8, 5))
sns.barplot(data=status_sales, x='STATUS', y='SALES', palette='Set3')
plt.title('Total Sales by Order Status')
plt.xlabel('Order Status')
plt.ylabel('Sales')
plt.tight_layout()
plt.show()
```

C:\Users\guard\AppData\Local\Temp\ipykernel_5940\3892669116.py:5: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.barplot(data=status_sales, x='STATUS', y='SALES', palette='Set3')
sns.barplot(data=pricebin_sales, x='PRICE_BIN', y='SALES', palette='plasma')
```



```
[40]: # Average Sales by Price Range
# Create price bins
df['PRICE_BIN'] = pd.cut(df['PRICEEACH'], bins=[0, 30, 60, 90, 120, 150, 200], labels=['0-30', '30-60', '60-90', '90-120', '120-150', '150-200'])

pricebin_sales = df.groupby('PRICE_BIN')['SALES'].mean().reset_index()

plt.figure(figsize=(8, 5))
sns.barplot(data=pricebin_sales, x='PRICE_BIN', y='SALES', palette='plasma')
plt.title('Average Sales per Order by Unit Price Range')
plt.xlabel('Unit Price Range')
plt.ylabel('Average Sales')
plt.tight_layout()
plt.show()
```

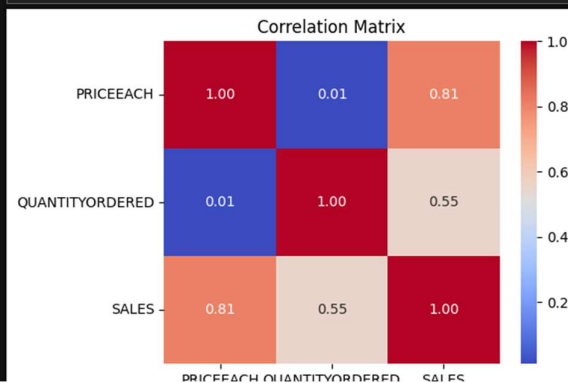
C:\Users\guard\AppData\Local\Temp\ipykernel_5940\1445162840.py:5: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
[39]: # Price vs Sales per Order
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='PRICEEACH', y='SALES', alpha=0.7)
plt.title('Unit Price vs Total Sales')
plt.xlabel('Unit Price')
plt.ylabel('Sales')
plt.grid(True)
plt.tight_layout()
plt.show()
```



```
# Correlation Between Key Metrics
# Check correlations
corr = df[['PRICEEACH', 'QUANTITYORDERED', 'SALES']].corr()

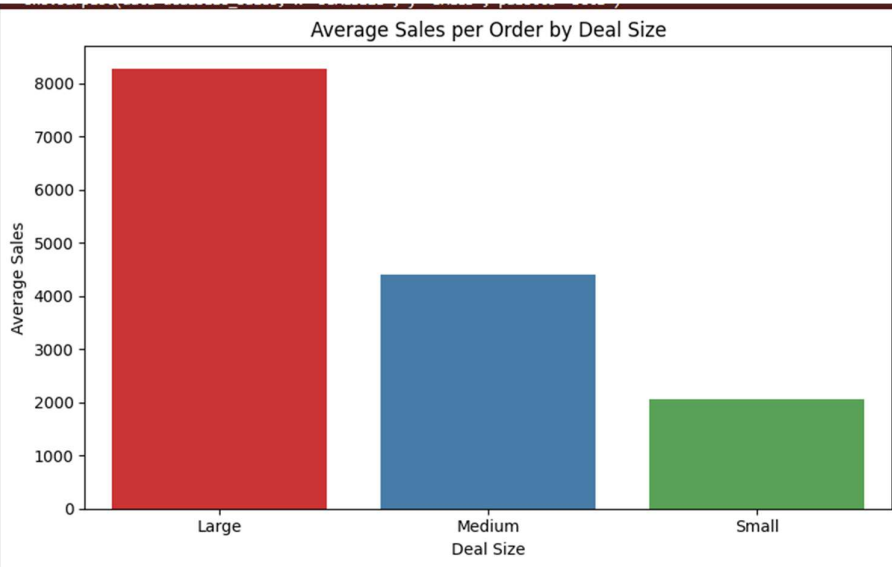
plt.figure(figsize=(8, 4))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix')
plt.tight_layout()
plt.show()
```



Objective 4: Pricing Strategy and Discount Impact

```
[36]: # Relationship Between Price and Quantity Ordered
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='PRICEEACH', y='QUANTITYORDERED', hue='DEALSIZE')
plt.title('Price vs Quantity Ordered')
plt.xlabel('Unit Price')
plt.ylabel('Quantity Ordered')
plt.grid(True)
plt.tight_layout()
plt.show()
```





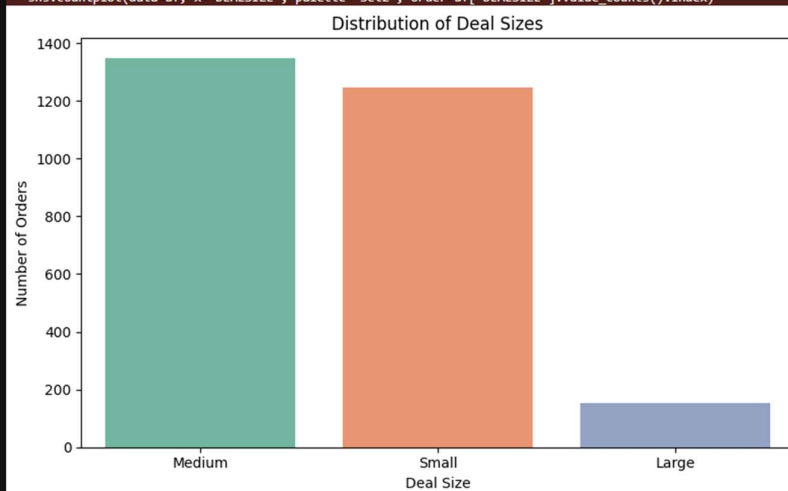
```
[35]: dealsize_sales = df.groupby('DEALSIZE')['SALES'].mean().reset_index().sort_values(by='SALES', ascending=False)
```

```
plt.figure(figsize=(8, 5))
sns.barplot(data=dealsize_sales, x='DEALSIZE', y='SALES', palette='Set1')
plt.title('Average Sales per Order by Deal Size')
plt.xlabel('Deal Size')
plt.ylabel('Average Sales')
plt.tight_layout()
plt.show()
```

C:\Users\guard\AppData\Local\Temp\ipykernel_5940\1703139726.py:4: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.barplot(data=dealsize_sales, x='DEALSIZE', palette='Set1', order=df['DEALSIZE'].value_counts().index)
```

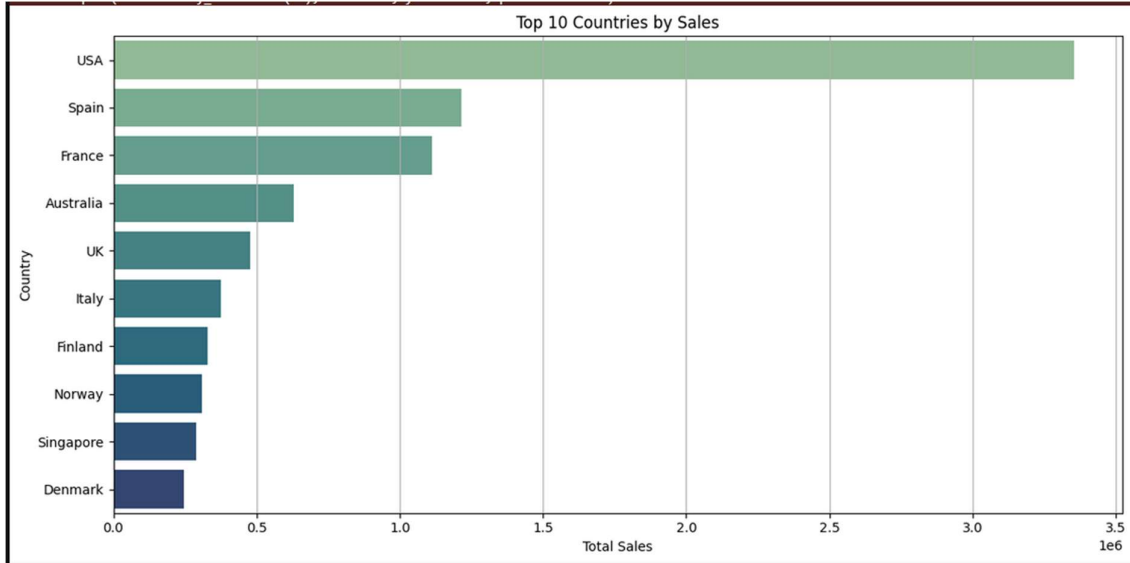
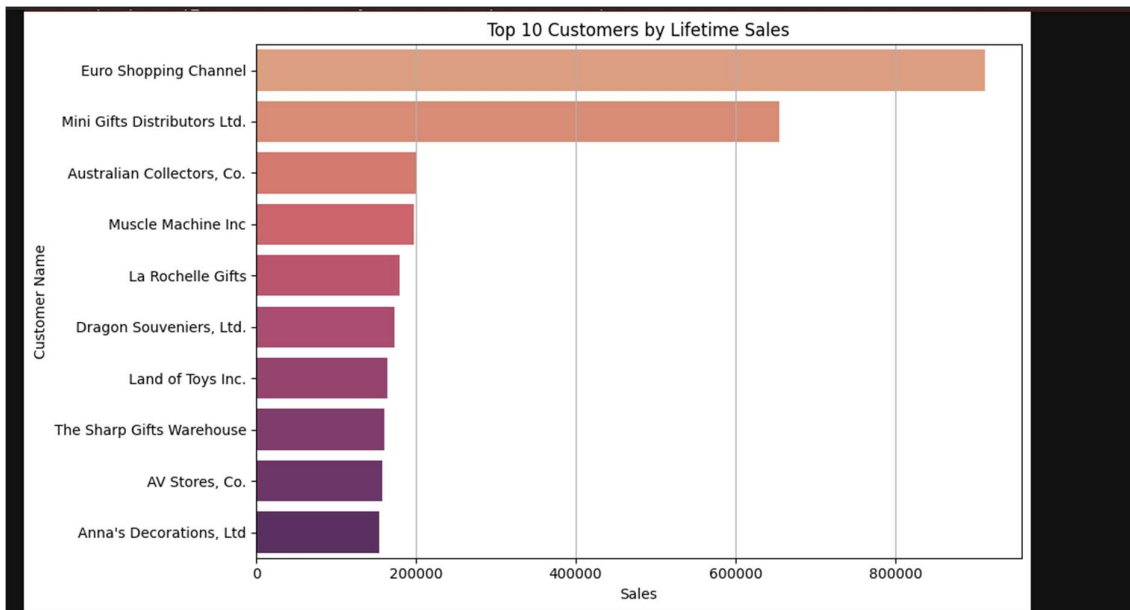


```
[34]: plt.figure(figsize=(8, 5))
sns.countplot(data=df, x='DEALSIZE', palette='Set2', order=df['DEALSIZE'].value_counts().index)
plt.title('Distribution of Deal Sizes')
plt.xlabel('Deal Size')
plt.ylabel('Number of Orders')
plt.tight_layout()
plt.show()
```

C:\Users\guard\AppData\Local\Temp\ipykernel_5940\3922947007.py:2: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.countplot(data=df, x='DEALSIZE', palette='Set2', order=df['DEALSIZE'].value_counts().index)
```



Objective 3: Customer Segmentation and Behavior

```
[32]: # Customer Count and Sales by Country
country_stats = df.groupby('COUNTRY').agg([
    'CUSTOMERNAME': 'nunique',
    'SALES': 'sum'
]).rename(columns={'CUSTOMERNAME': 'UNIQUE_CUSTOMERS'}).sort_values(by='SALES', ascending=False).reset_index()

plt.figure(figsize=(12, 6))
sns.barplot(data=country_stats.head(10), x='SALES', y='COUNTRY', palette='crest')
plt.title('Top 10 Countries by Sales')
plt.xlabel('Total Sales')
plt.ylabel('Country')
plt.grid(True, axis='x')
plt.tight_layout()
plt.show()

country_stats.head()
```

C:\Users\guard\AppData\Local\Temp\ipykernel_5940\3123431038.py:8: FutureWarning:

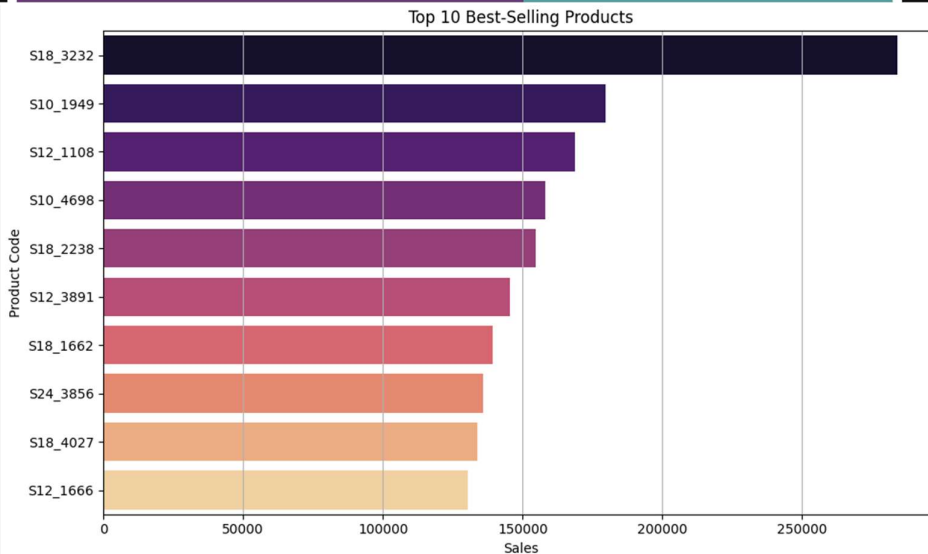
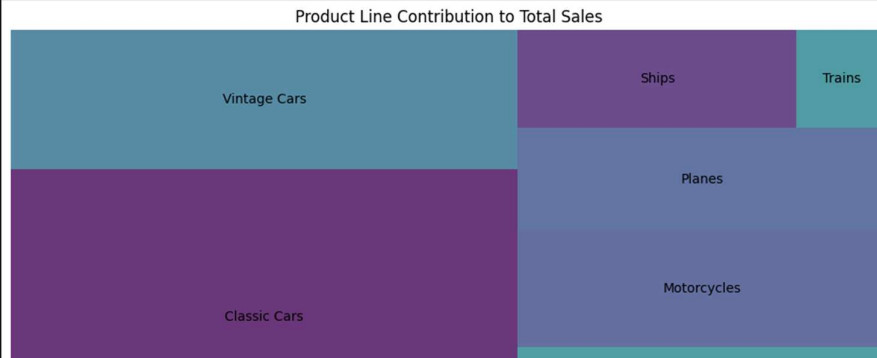
Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'y' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.barplot(data=country_stats.head(10), x='SALES', y='COUNTRY', palette='crest')
```



```
[31]: # Treemap of Product Line Contribution
# If not installed: !pip install squarify
import squarify

plt.figure(figsize=(12, 6))
squarify.plot(sizes=productline_sales['SALES'], label=productline_sales['PRODUCTLINE'], alpha=.8)
plt.title('Product Line Contribution to Total Sales')
plt.axis('off')
plt.show()
```



```
[29]: # Average Price and Quantity by Product Line
productline_stats = df.groupby("PRODUCTLINE").agg({
    'PRICEEACH': 'mean',
    'QUANTITYORDERED': 'mean',
    'SALES': 'mean'
}).reset_index()

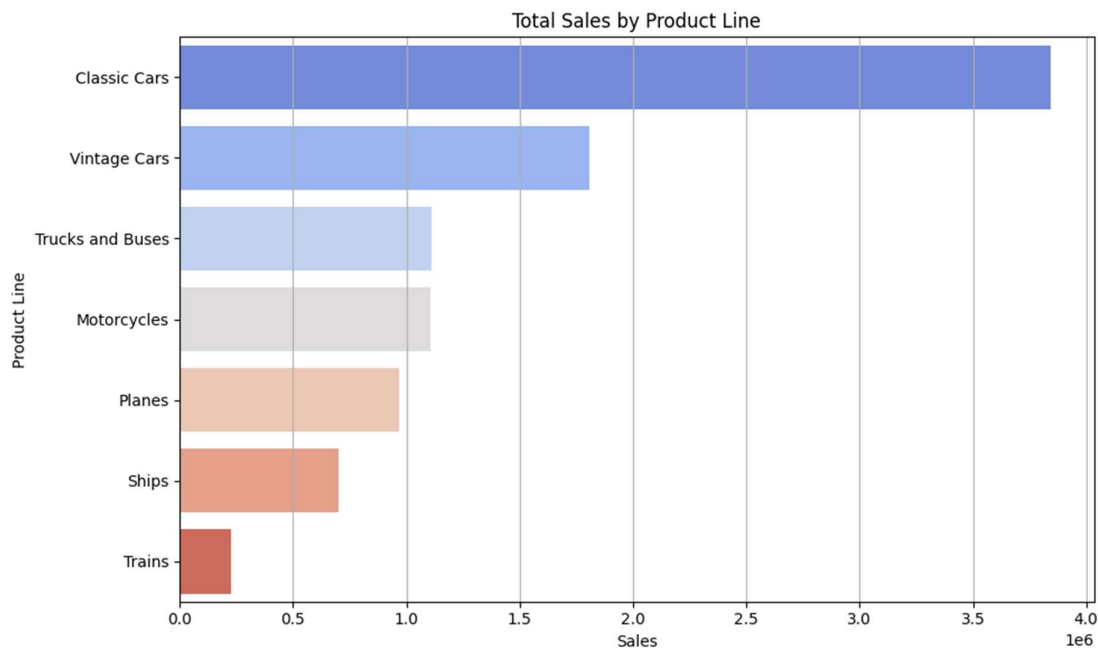
print(productline_stats)
```

	PRODUCTLINE	PRICEEACH	QUANTITYORDERED	SALES
0	Classic Cars	115.195680	35.166491	4049.387292
1	Motorcycles	99.767125	35.399361	3525.598051
2	Planes	90.517829	34.986842	3188.563882
3	Ships	88.169261	34.734783	3043.648783
4	Trains	84.108701	35.220779	2938.226883
5	Trucks and Buses	104.344983	35.861017	3767.997254
6	Vintage Cars	90.011261	34.644214	3120.337962

```
[30]: # Top 10 Best-Selling Products (by SALES)
top_products = df.groupby("PRODUCTCODE")["SALES"].sum().reset_index().sort_values(by="SALES", ascending=False).head(10)

plt.figure(figsize=(10, 6))
sns.barplot(data=top_products, x="SALES", y="PRODUCTCODE", palette="magma")
plt.title('Top 10 Best-Selling Products')
plt.xlabel('Sales')
plt.ylabel('Product Code')
plt.grid(True, axis='x')
plt.tight_layout()
plt.show()
```

C:\Users\guard\AppData\Local\Temp\ipykernel_5940\14660599.py:5: FutureWarning:



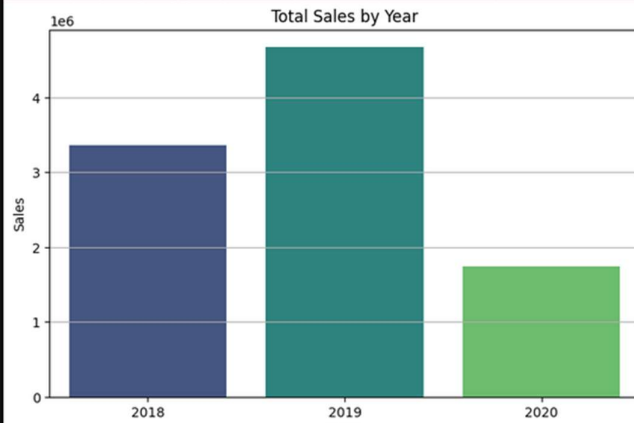
```
[27]: # Total sales by year
yearly_sales = df.groupby('YEAR')['SALES'].sum().reset_index()

plt.figure(figsize=(8, 5))
sns.barplot(data=yearly_sales, x='YEAR', y='SALES', palette='viridis')
plt.title('Total Sales by Year')
plt.ylabel('Sales')
plt.grid(True, axis='y')
plt.show()
```

C:\Users\guard\AppData\Local\Temp\ipykernel_5940\2503783288.py:5: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.barplot(data=yearly_sales, x='YEAR', y='SALES', palette='viridis')
```



```
[26]: #Plot Monthly Sales Trends
plt.figure(figsize=(12, 6))
sns.lineplot(data=monthly_sales, x='MONTH_NAME', y='SALES', hue='YEAR', marker='o')
plt.title('Monthly Sales Trends by Year')
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```



Objective 1: Sales Trend Analysis

```
[24]: # Create Year and Month columns
df['YEAR'] = df['ORDERDATE'].dt.year
df['MONTH'] = df['ORDERDATE'].dt.month
df['MONTH_NAME'] = df['ORDERDATE'].dt.strftime('%B')
```

```
[25]: # Group by Year and Month for total sales
monthly_sales = df.groupby(['YEAR', 'MONTH_NAME'])['SALES'].sum().reset_index()

# To maintain calendar order
from pandas.api.types import CategoricalDtype
month_order = ['January', 'February', 'March', 'April', 'May', 'June',
               'July', 'August', 'September', 'October', 'November', 'December']
month_type = CategoricalDtype(categories=month_order, ordered=True)
monthly_sales['MONTH_NAME'] = monthly_sales['MONTH_NAME'].astype(month_type)

# Sort for proper plotting
monthly_sales = monthly_sales.sort_values(['YEAR', 'MONTH_NAME'])
```

```
[26]: #Plot Monthly Sales Trends
plt.figure(figsize=(12, 6))
sns.lineplot(data=monthly_sales, x='MONTH_NAME', y='SALES', hue='YEAR', marker='o')
```

```
PHONE 0
ADDRESSLINE1 0
CITY 0
POSTALCODE 0
COUNTRY 0
CONTACTLASTNAME 0
CONTACTFIRSTNAME 0
DEALSIZE 0
dtype: int64

[22]: # Convert ORDERDATE to datetime
df['ORDERDATE'] = pd.to_datetime(df['ORDERDATE'], dayfirst=True)

[23]: # Unique product lines
print("Product Lines:", df['PRODUCTLINE'].unique())

# Status distribution
print("Order Statuses:\n", df['STATUS'].value_counts())

# Country-wise customer count
print("Customers by Country:\n", df['COUNTRY'].value_counts())

Product Lines: ['Motorcycles' 'Classic Cars' 'Trucks and Buses' 'Vintage Cars' 'Planes'
'Ships' 'Trains']
Order Statuses:
STATUS
4 10168 36 96.66 1 3479.76 28/10/2018 586 Shipped Motorcycles 95

[21]: # Dataset info
df.info()

# Summary statistics
df.describe()

# Check for missing values
df.isnull().sum()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2747 entries, 0 to 2746
Data columns (total 20 columns):
# Column Non-Null Count Dtype
---
0 ORDERNUMBER 2747 non-null int64
1 QUANTITYORDERED 2747 non-null int64
2 PRICEEACH 2747 non-null float64
3 ORDERLINENUMBER 2747 non-null int64
4 SALES 2747 non-null float64
5 ORDERDATE 2747 non-null object
6 DAYS_SINCE_LASTORDER 2747 non-null int64
```

5. Conclusion

This project demonstrates how structured data analysis can provide critical business insights into auto sales operations. From understanding seasonal sales trends to identifying top-performing products and customers, the findings support more data-driven decisions. The analyses reveal not only what has happened but provide guidance for strategy moving forward in terms of marketing, inventory planning, and customer management.

6. Future Scope

There is substantial potential to build upon this analysis using advanced analytics and predictive modeling. Future developments may include:

- Implementing machine learning models to forecast monthly or quarterly sales.
- Building customer segmentation models to tailor marketing strategies.

- Analyzing customer churn and satisfaction using order history and status data.
 - Integrating external datasets like economic indicators to enrich insights.
 - Creating real-time dashboards using BI tools like Power BI or Tableau for continuous monitoring.
-

7. References

- Dataset Source: Auto Sales data.csv (public dataset)
- Python Libraries Used: pandas, numpy, matplotlib, seaborn, squarify
- Development Environment: Jupyter Notebook
- Seaborn Documentation: <https://seaborn.pydata.org/>
- Matplotlib Documentation: <https://matplotlib.org/>
- Pandas Documentation: <https://pandas.pydata.org/>
- Additional Online Resources and Tutorials for Data Analysis with Python